ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

Departamento de Ingeniería

Trabajo Práctico 01

Ejercicio 2: Coloración de Grafos con k Colores

Asignatura: Análisis y Diseño de Algoritmos
Profesor: Carlos Alberto Álvarez Henao
Integrantes: Andrés Felipe Vélez Álvarez
Sebastián Salazar Henao
Nathalia Valentina Cardoza Azuaje
Fecha: 5 de abril de 2026

Universidad EAFIT

1. Descripción teórica

El problema de k**-coloración** consiste en asignar un color a cada vértice de un grafo no dirigido $G = (V, E)$ usando colores del conjunto $\{1, \ldots, k\}$, cumpliendo que vértices adyacentes tengan distinto color:

$$\forall (u, v) \in E : \quad \text{color}[u] \neq \text{color}[v].$$

Relación con número cromático:

$$G \text{ es } k\text{-coloreable} \iff \chi(G) \leq k.$$

2. Enfoques de solución

2.1. Backtracking

El enfoque de backtracking construye la coloración de forma incremental, asignando color al vértice v y avanzando al vértice $v + 1$. Antes de avanzar, valida si el color asignado es compatible con los vecinos ya coloreados mediante la función `es_seguro`. Si se detecta un conflicto, esa rama se descarta inmediatamente (*poda*) y no se sigue explorando.

Este enfoque permite:

- encontrar la primera solución válida;
- contar todas las soluciones válidas;
- reducir significativamente el espacio explorado cuando el grafo es denso o cuando k es pequeño.

En el código `main_ejercicio2.cpp`, además de contar soluciones, se guarda la primera coloración válida y se lleva el conteo de nodos explorados para medir la efectividad de la poda.

2.2. Fuerza bruta

El enfoque de fuerza bruta genera todas las asignaciones posibles de colores para n vértices, es decir, k^n combinaciones. Cada combinación se verifica completa para comprobar si cumple la restricción de coloración.

En esta estrategia no existe poda: aun cuando una combinación ya es inválida en los primeros vértices, se termina de generar y luego se valida. Por eso su costo crece rápidamente y se usa principalmente como referencia para comparar rendimiento y para verificar que el conteo de backtracking sea correcto.

3. Pseudocódigo completo

Listing 1: Backtracking

```
leer n, k
leer adj[0..n-1][0..n-1]      // matriz de adyacencia
color[0..n-1] = 0             // 0 = sin color
```

```
total_soluciones = 0
primera_guardada = falso
nodos_bt = 0

es_seguro(v,c):
  para u en [0..n-1]:
    si adj[v][u] == 1 y color[u] == c:
      retornar falso
  retornar verdadero

bt(v):
  nodos_bt += 1
  si v == n:
    total_soluciones += 1
    si no primera_guardada:
      primera_solucion = copia(color)
      primera_guardada = verdadero
    retornar

  para c en [1..k]:
    si es_seguro(v, c):
      color[v] = c
      bt(v + 1)
      color[v] = 0            // backtrack

bt(0)

si total_soluciones == 0:
  imprimir "No existe una k-coloracion valida"
si no:
  imprimir "Total de coloraciones validas:", total_soluciones
  imprimir "Primera solucion:", primera_solucion
imprimir "Nodos explorados:", nodos_bt
```

Listing 2: Fuerza bruta

```
total_fb = 0
nodos_fb = 0
total_comb = k^n

verificar_coloracion(col):
  para u en [0..n-1]:
    para v en [u+1..n-1]:
      si adj[u][v] == 1 y col[u] == col[v]:
        retornar falso
  retornar verdadero

para mask en [0..total_comb-1]:
  nodos_fb += 1
  tmp = mask
  para v en [0..n-1]:
    col[v] = (tmp mod k) + 1
```

```
    tmp = tmp div k

  si verificar_coloracion(col):
    total_fb += 1

imprimir "Total de combinaciones:", total_comb
imprimir "Coloraciones validas (fuerza bruta):", total_fb
imprimir "Nodos explorados:", nodos_fb
```

4. Implementación en C++

El archivo principal es `main_ejercicio2.cpp`. Implementa: lectura de matriz de adyacencia, conteo por backtracking, conteo por fuerza bruta, primera solución válida y comparación de tiempos.

La medición usa `REPS = 100000` para reportar tiempo promedio en microsegundos.

5. Resultados experimentales (salida real)

Los datos siguientes corresponden a la opción 2 del programa (benchmarks predefinidos).

5.1. Caso 1: Ciclo C_4 con $k = 3$

En este caso sí existen múltiples coloraciones válidas. Backtracking encuentra 18 soluciones, explora 40 nodos y la primera asignación reportada por el programa es $1, 2, 1, 2$. Fuerza bruta también encuentra 18 soluciones, pero recorre las 81 combinaciones posibles.

```
--------------------------------------------
  CASO: Ciclo C4  |  n=4, k=3
--------------------------------------------

============================================
           RESULTADO: BACKTRACKING
============================================
  Coloraciones validas encontradas : 18
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 2
    vertice 2  ->  color 1
    vertice 3  ->  color 2
  Nodos explorados    : 40
  Tiempo de ejecucion : 0.7620 us  (promedio de 100000 repeticiones)

============================================
           RESULTADO: FUERZA BRUTA
============================================
  Combinaciones totales generadas  : 81  (3^4)
  Coloraciones validas encontradas : 18
  Tiempo de ejecucion : 4.9232 us  (promedio de 100000 repeticiones)

============================================
            VERIFICACION CRUZADA
============================================
  OK  Ambos enfoques coinciden: 18 coloraciones validas.

============================================
         COMPARACION DE EFICIENCIA
============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                    40   0.7620 us
  Fuerza Bruta                    81   4.9232 us

  La poda del backtracking elimino el 50.62% de los nodos.

  Grafo 2: Completo K4 (n=4, k=4)
  Aristas: todos los pares posibles
```

5.2. Caso 2: Grafo completo K_4 con $k = 4$

Al ser un grafo denso, las restricciones aparecen muy temprano. Backtracking reporta 24 coloraciones válidas y explora 65 nodos, mientras que fuerza bruta debe evaluar 256 combinaciones. La primera solución mostrada por el programa es $1, 2, 3, 4$.

```
--------------------------------------------
  CASO: Completo K4  |  n=4, k=4
--------------------------------------------

============================================
           RESULTADO: BACKTRACKING
============================================
  Coloraciones validas encontradas : 24
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 2
    vertice 2  ->  color 3
    vertice 3  ->  color 4
  Nodos explorados    : 65
  Tiempo de ejecucion : 0.7756 us  (promedio de 100000 repeticiones)

============================================
           RESULTADO: FUERZA BRUTA
============================================
  Combinaciones totales generadas  : 256  (4^4)
  Coloraciones validas encontradas : 24
  Tiempo de ejecucion : 8.4628 us  (promedio de 100000 repeticiones)

============================================
            VERIFICACION CRUZADA
============================================
  OK  Ambos enfoques coinciden: 24 coloraciones validas.

============================================
         COMPARACION DE EFICIENCIA
============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                   65   0.7756 us
  Fuerza Bruta                  256   8.4628 us

  La poda del backtracking elimino el 74.61% de los nodos.

  Grafo 3: C7 + cuerdas (n=7, k=3)
  Ciclo de 7 vertices con cuerdas adicionales
```

5.3. Caso 3: C_7 con cuerdas y $k = 3$

Este caso no tiene 3-coloración válida. Backtracking detecta conflictos y poda rápidamente, por eso solo explora 34 nodos. En contraste, fuerza bruta revisa las 2187 combinaciones y confirma también que el total de soluciones es 0.

```
--------------------------------------------
  CASO: C7+cuerdas  |  n=7, k=3
--------------------------------------------

============================================
           RESULTADO: BACKTRACKING
============================================
  No existe una 3-coloracion valida para este grafo.
  Nodos explorados    : 34
  Tiempo de ejecucion : 0.5754 us  (promedio de 100000 repeticiones)

============================================
           RESULTADO: FUERZA BRUTA
============================================
  Combinaciones totales generadas  : 2187  (3^7)
  Coloraciones validas encontradas : 0
  Tiempo de ejecucion : 112.2828 us  (promedio de 100000 repeticiones)

============================================
            VERIFICACION CRUZADA
============================================
  OK  Ambos enfoques coinciden: 0 coloraciones validas.

============================================
         COMPARACION DE EFICIENCIA
============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                   34   0.5754 us
  Fuerza Bruta                 2187   112.2828 us

  La poda del backtracking elimino el 98.45% de los nodos.

  Grafo 4: Bipartito K2,3 (n=5, k=2)
  Vertices {0,1} conectados con {2,3,4}
```

5.4. Caso 4: Bipartito $K_{2,3}$ con $k = 2$

El grafo sí es 2-coloreable y el programa encuentra 2 coloraciones válidas. Backtracking explora 13 nodos y entrega como primera solución $1, 1, 2, 2, 2$. Fuerza bruta valida el mismo

total con 32 combinaciones.

```
--------------------------------------------
  CASO: Bipartito K2,3  |  n=5, k=2
--------------------------------------------

============================================
           RESULTADO: BACKTRACKING
============================================
  Coloraciones validas encontradas : 2
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 1
    vertice 2  ->  color 2
    vertice 3  ->  color 2
    vertice 4  ->  color 2
  Nodos explorados    : 13
  Tiempo de ejecucion : 0.0900 us  (promedio de 100000 repeticiones)

============================================
           RESULTADO: FUERZA BRUTA
============================================
  Combinaciones totales generadas  : 32  (2^5)
  Coloraciones validas encontradas : 2
  Tiempo de ejecucion : 1.0616 us  (promedio de 100000 repeticiones)

============================================
            VERIFICACION CRUZADA
============================================
  OK  Ambos enfoques coinciden: 2 coloraciones validas.

============================================
         COMPARACION DE EFICIENCIA
============================================
  Metodo             Nodos explorados   Tiempo [us]
  Backtracking                     13   0.0900 us
  Fuerza Bruta                     32   1.0616 us

  La poda del backtracking elimino el 59.38% de los nodos.
```

5.5. Resumen comparativo

Tabla 1: Comparación: Backtracking vs Fuerza Bruta

Grafo	n,k	Nodos BT	Nodos FB	T. BT [µs]	T. FB [µs]	Poda
Ciclo C_4	4,3	40	81	0.7620	4.9232	50.62 %
Completo K_4	4,4	65	256	0.7756	8.4628	74.61 %
C_7 + cuerdas	7,3	34	2187	0.5754	112.2828	98.45 %
Bipartito $K_{2,3}$	5,2	13	32	0.0900	1.0616	59.38 %

En los cuatro casos, ambos enfoques coinciden en el número de soluciones, pero backtracking explora menos nodos y obtiene mejor tiempo por la poda.

6. Análisis de complejidad

6.1. Complejidad temporal

Fuerza bruta

Este método genera k^n combinaciones. Para cada combinación, la función de verificación revisa pares de vértices adyacentes con la matriz de adyacencia, lo cual en el peor caso cuesta $O(n^2)$. Por tanto:

$$T_{FB}(n,k) = O(k^n \cdot n^2), \qquad S_{FB}(n) = O(n^2).$$

Backtracking

En el peor caso (por ejemplo, grafos con poca restricción), el árbol de búsqueda puede acercarse a k^n asignaciones. En cada intento de asignación, `es_seguro` recorre hasta n vecinos por usar matriz de adyacencia. Una cota superior práctica para esta implementación

es:

$$T_{BT}(n, k) = O(k^n \cdot n).$$

Aunque sigue siendo exponencial, en la práctica suele ser mucho menor por la poda temprana de ramas inválidas.

6.2. Complejidad espacial

Ambos métodos comparten el almacenamiento principal: matriz de adyacencia $n \times n$, que domina con $O(n^2)$. Además, se almacenan arreglos de colores de tamaño $O(n)$.

En backtracking se agrega la pila de recursión de profundidad máxima n, equivalente a $O(n)$. Entonces, para ambos enfoques, la complejidad espacial total queda dominada por:

$$S(n) = O(n^2).$$

En síntesis: la diferencia clave entre enfoques no está en memoria, sino en tiempo de ejecución, donde backtracking aprovecha la poda.

7. Respuestas guía

1. **¿Cuántos nodos explora Backtracking vs Fuerza Bruta?**

En el ejemplo C_4 con $k = 3$, backtracking explora 40 nodos y fuerza bruta 81. La diferencia se debe a que backtracking descarta ramas apenas aparece una asignación inválida, mientras que fuerza bruta genera cada combinación completa antes de verificar.

2. **¿Qué tipo de grafo maximiza o minimiza la poda?**

La poda se maximiza en grafos densos (por ejemplo, K_n) porque hay muchas aristas y, por tanto, más probabilidad de conflicto temprano. Se minimiza en grafos muy dispersos o sin aristas, donde casi cualquier asignación parcial es válida y el árbol de búsqueda se recorre casi completo.

3. **¿Qué cambio hacer si solo se quiere saber si existe una coloración?**

Se convierte el algoritmo de conteo en uno de decisión: cuando se encuentra la primera solución completa, la función retorna `true` y propaga ese resultado para terminar la recursión sin seguir explorando ramas.

4. **Relación con el número cromático.**

El número cromático $\chi(G)$ es el mínimo número de colores necesario para colorear un grafo sin conflictos. Por eso, resolver el problema de k-coloración equivale a verificar si $\chi(G) \leq k$. En la práctica, puede estimarse $\chi(G)$ probando valores crecientes de k hasta encontrar el primero que produzca al menos una solución válida.

8. Conclusiones

- La implementación confirma la corrección de ambos enfoques: en todos los casos, backtracking y fuerza bruta reportan el mismo total de coloraciones válidas.

- Backtracking fue consistentemente más eficiente, tanto en nodos explorados como en tiempo de ejecución, debido a la poda de ramas inválidas.
- La efectividad de la poda depende de la estructura del grafo. En este trabajo varió entre 50.62 % y 98.45 %, mostrando mayor impacto en instancias más restrictivas.
- El caso C_7 con cuerdas muestra con claridad la diferencia de enfoques: cuando no existe solución, backtracking puede detectar conflictos pronto, mientras fuerza bruta mantiene el costo de evaluar todo el espacio k^n.
- Fuerza bruta sigue siendo útil como línea base y como mecanismo de verificación cruzada del conteo obtenido por backtracking.
- Aunque la poda mejora el rendimiento práctico, ambos algoritmos tienen crecimiento exponencial para instancias grandes. Para grafos de mayor tamaño conviene estudiar heurísticas de ordenamiento de vértices y métodos aproximados.